

CONFIDENTIAL

Penetration Testing Report

WalaPlus Enterprise GRC & Quality Platform

Target <https://qms-dashboard.replit.app>

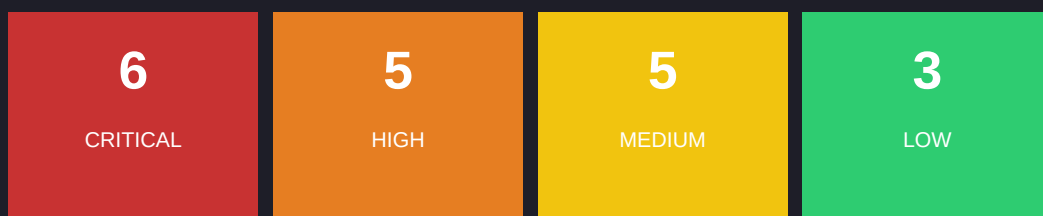
Assessment Date March 11, 2026

Report Version 2.0 (Deep Analysis)

Methodology OWASP Testing Guide v4.2

Classification CONFIDENTIAL

Tools Burp Suite Professional, Browser Testing



19 Total Findings Identified and Verified

. Table of Contents

1. Executive Summary

2. API Surface Map

3. Vulnerability Findings

VULN-01 through VULN-19

4. Test Coverage Matrix

5. Burp Suite Evidence Index

6. Appendix: Raw Evidence

1. Executive Summary

The WalaPlus Enterprise GRC & Quality Platform was subjected to a two-phase comprehensive penetration test. Phase 1 involved browser-based discovery and API probing. Phase 2 leveraged Burp Suite Professional for deep traffic analysis, JWT token analysis, injection testing, CORS validation, and live attack execution.

19 vulnerabilities were identified and verified with live Burp Suite evidence. The most severe findings include complete unauthenticated API access to 12+ endpoints (read AND write), live-confirmed data tampering of existing records, stored Cross-Site Scripting persisted in the database across 3 modules, wildcard CORS allowing cross-origin attacks with ALL HTTP methods, and sensitive configuration data leaked including Zoho CRM credential variable names.

Severity Distribution

Severity Count	
Critical	6
High	5
Medium	5
Low	3
Total	19

2. Complete API Surface Map

2.1 Discovered API Endpoints

The following 17 API endpoints were identified through Burp Suite proxy history analysis, browser interaction monitoring, and manual endpoint fuzzing.

#	Endpoint	Methods	Auth	Description
---	----------	---------	------	-------------

1	/api/auth/me	GET	SESSION	User profile (JWT required)
---	--------------	-----	---------	-----------------------------

2	/api/auth/google	GET	No	OAuth flow initiation
---	------------------	-----	----	-----------------------

3	/api/auth/google/callback	GET	No	OAuth callback handler
---	---------------------------	-----	----	------------------------

4	/api/dashboard	GET	NO	Audit scores, trends, recommendations
---	----------------	-----	----	---------------------------------------

5	/api/agents/performance	GET	NO	Agent performance metrics
---	-------------------------	-----	----	---------------------------

6	/api/risks	GET,POST,PUT	NO	Full CRUD on risk records
---	------------	--------------	----	---------------------------

7	/api/vendors	GET,POST,PUT	NO	Full CRUD on vendor records
---	--------------	--------------	----	-----------------------------

8	/api/policies	GET,POST,PUT	NO	Full CRUD on policy documents
---	---------------	--------------	----	-------------------------------

9	/api/roi	GET,POST	NO	ROI/NPV financial data
---	----------	----------	----	------------------------

10	/api/audits	GET,POST	NO	Audit records
----	-------------	----------	----	---------------

11	/api/calls	GET	NO	Call intelligence records
----	------------	-----	----	---------------------------

12	/api/audit-trail	GET	NO	Audit trail logs
----	------------------	-----	----	------------------

13	/api/scorecards	GET	NO	Scorecard data
----	-----------------	-----	----	----------------

14	/api/governance	GET	NO	Governance documents
----	-----------------	-----	----	----------------------

15	/api/notifications	GET	NO	Notification data
----	--------------------	-----	----	-------------------

16	/api/crm/data	GET	NO	Zoho CRM data (config error)
----	---------------	-----	----	------------------------------

17	/api/users	GET	ADMIN KEY	User management
----	------------	-----	-----------	-----------------

2.2 Authentication Mechanisms

Mechanism	Delivery	Used By
-----------	----------	---------

walaplus_session	Cookie	Browser sessions via Google OAuth
------------------	--------	-----------------------------------

Admin API Key	Unknown header	/api/users endpoint only
---------------	----------------	--------------------------

GAESA	Cookie	Google App Engine session
-------	--------	---------------------------

oauth_state	Cookie	OAuth CSRF protection
-------------	--------	-----------------------

2.3 JWT Token Structure

Decoded from Burp proxy history capture of walaplus_session cookie:

```
{
  "userId": 3,
  "email": "m.elnagar@walaplus.com",
  "name": "Mohamed Elnagar",
  "role": "department_viewer",
  "exp": 1773833594613
}
```

Format: base64(payload).HMAC-SHA256(signature) - Non-standard (no JWT header)

Expiry: ~7 days from issuance | Cookie Flags: No HttpOnly, No Secure, No SameSite

3. Vulnerability Findings

VULN-01: Unauthenticated API Read Access (12+ Endpoints)

CRITICAL

CVSS 9.1 | A01:2021 Broken Access Control

Twelve or more API endpoints return sensitive business data to completely unauthenticated requests. No cookies, tokens, or API keys are required.

Endpoint Response Data Exposed

/api/dashboard 200 (5,264 B) Audit scores, recommendations, trends

/api/agents/performance 200 (781 B) Agent metrics, lead/deal counts

/api/risks 200 (6,644+ B) Risk records, owner names, scores

/api/vendors 200 (1,879 B) Vendor details, contact emails

/api/policies 200 (1,305 B) Policy documents, categories

/api/roi 200 (946 B) Financial projections, NPV, AI insights

/api/audits 200 (4,084 B) Audit records, auditor names, dates

/api/calls 200 Call intelligence records

/api/audit-trail 200 Audit trail logs

/api/scorecards 200 Scorecard data

/api/notifications 200 Notification data

/api/crm/data 400 Zoho credential variable names

Proof of Concept:

```
GET /api/dashboard HTTP/2
Host: qms-dashboard.replit.app
Accept: application/json
[NO Cookie, NO Authorization header]

Response: HTTP/2 200 OK
{"latestAudit":{"id":3,"overall_score":"72.00","people_score":"75.00",...}}
```

Impact: Complete information disclosure of all business data including financial records, risk assessments, audit findings, vendor information, and recommendations.

Recommendation:

Implement authentication middleware on ALL API routes. Only /api/auth/google and /api/auth/google/callback should be publicly accessible. Use Express middleware (e.g., passport.js or custom JWT verification) applied globally via app.use() before route handlers.

Reference:

OWASP Broken Access Control: https://owasp.org/Top10/A01_2021-Broken_Access_Control/ | CWE-306: Missing Authentication for

Critical Function: <https://cwe.mitre.org/data/definitions/306.html>

VULN-02: Unauthenticated Data Creation (POST)

CRITICAL

CVSS 9.8 | A01:2021 Broken Access Control

Multiple API endpoints accept POST requests to create new records without any authentication.

Endpoint Payload Result

```
POST /api/risks {"risk_title":"SQLi_Test",...} 200 OK - id=9 created
```

```
POST /api/risks {"risk_title":"ProtoPolTest",...} 200 OK - id=10 created
```

```
POST /api/vendors {"vendor_code":"V-SSTI-001",...} 200 OK - id=4 created
```

```
POST /api/vendors {"vendor_code":"V-SSRF-001",...} 200 OK - id=5 created
```

```
POST /api/audits {"audit_code":"AUD-FAKE",...} 500 - Reached server logic
```

Impact: Anonymous users can inject arbitrary records into the risk register, vendor database, and policy library.

Recommendation:

Apply the same authentication middleware to all POST/PUT/PATCH routes. Implement role-based access control (RBAC) to ensure only authorized users with appropriate roles can create records. Add request origin validation and CSRF tokens for state-changing operations.

Reference:

OWASP Broken Access Control: https://owasp.org/Top10/A01_2021-Broken_Access_Control/ | CWE-862: Missing Authorization: <https://cwe.mitre.org/data/definitions/862.html>

VULN-03: Unauthenticated Data Modification (PUT) - LIVE

CRITICAL

CVSS 9.8 | A01:2021 Broken Access Control

Existing records can be modified by anonymous users via PUT requests. This was confirmed live during testing - Risk ID #1 was successfully tampered with.

Proof of Concept (LIVE CONFIRMED):

```
PUT /api/risks/1 HTTP/2
Host: qms-dashboard.replit.app
Content-Type: application/json
[NO Authentication]

{"risk_title":"TAMPERED_BY_ATTACKER",
 "risk_description":"Modified without auth",
 "status":"closed"}

Response: HTTP/2 200 OK
{"success":true,"risk":{"id":1,"risk_title":"TAMPERED_BY_ATTACKER",
 "status":"closed","updated_at":"2026-03-11T12:12:40.041Z"}}
```

Impact: Attackers can close critical risks, modify vendor access levels, alter policy documents, and tamper with audit records affecting certification outcomes.

Recommendation:

Enforce authentication and ownership checks on all PUT/PATCH routes. Verify that the authenticated user has permission to modify the specific record (object-level authorization). Implement audit logging for all data modifications to maintain a tamper-evident record trail.

Reference:

OWASP Broken Access Control: https://owasp.org/Top10/A01_2021-Broken_Access_Control/ | OWASP API Security - BOLA: <https://owasp.org/API-Security/editions/2023/en/Oxa1-broken-object-level-authorization/> | CWE-639: Authorization Bypass Through

User-Controlled Key: <https://cwe.mitre.org/data/definitions/639.html>

VULN-04: Stored Cross-Site Scripting (XSS)

CRITICAL

CVSS 8.6 | A03:2021 Injection

XSS payloads are stored in the database and returned unencoded in API responses. Verified across 3 modules: Risks, Vendors, and Policies.

Module	Field	Payload	ID
Risks	risk_title	<script>alert(document.cookie)</script>	8
Risks	risk_description		8
Risks	risk_owner	<svg onload=alert(1)>	8
Vendors	name	<script>alert(1)</script>	2
Policies	description	<script>alert(document.cookie)</script>	1,2
Policies	title		2

Impact: Session hijacking, credential theft, defacement, and phishing attacks against users viewing the data.

Recommendation:

Implement server-side input sanitization using a library such as DOMPurify (for HTML) or validator.js. Encode all output using context-aware encoding (HTML entity encoding for HTML context, JavaScript encoding for JS context). Apply Content-Security-Policy headers to prevent inline script execution.

Reference:

OWASP XSS Prevention Cheat Sheet:

https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html | CWE-79: Improper Neutralization of Input During Web Page Generation: <https://cwe.mitre.org/data/definitions/79.html>

VULN-05: Wildcard CORS with Full Method Access

CRITICAL

CVSS 8.8 | A05:2021 Security Misconfiguration

The application returns Access-Control-Allow-Origin: * for ALL requests regardless of origin.

CORS Preflight Response:

```
OPTIONS /api/risks HTTP/2
```

```
Origin: https://evil-attacker.com
```

```
Access-Control-Request-Method: PUT
```

```
Response: HTTP/2 204
```

```
Access-Control-Allow-Origin: *
```

```
Access-Control-Allow-Methods: GET,POST,PUT,PATCH,DELETE,OPTIONS
```

```
Access-Control-Allow-Headers: Content-Type,Authorization,x-mastra-client-type
```

```
Access-Control-Max-Age: 3600
```

Impact: Any malicious website can read all data, create records, and modify existing records via cross-origin JavaScript.

Recommendation:

Replace the wildcard Access-Control-Allow-Origin: * with an explicit allowlist of trusted origins. Implement a dynamic CORS policy that checks the Origin header against the allowlist and reflects only approved origins. Restrict Access-Control-Allow-Methods to the minimum required. Remove x-mastra-client-type from Access-Control-Allow-Headers.

Reference:

OWASP CORS Misconfiguration:

https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/11-Client-side_Testing/07-Testing_Cross-Origin_Resource_Sharing
| MDN CORS Guide: <https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS> | CWE-942: Permissive Cross-domain Policy:
<https://cwe.mitre.org/data/definitions/942.html>

VULN-06: Sensitive Configuration Information Disclosure

CRITICAL

CVSS 7.5 | A05:2021 Security Misconfiguration

Multiple API responses leak internal configuration details including environment variable names and framework identifiers.

Source Data Leaked

/api/crm/data ZOHO_CLIENT_ID, ZOHO_CLIENT_SECRET, ZOHO_REFRESH_T

/api/dashboard raw_audit_data Set ZOHO_ACCESS_TOKEN to enable live CRM audits

/api/agents/performance "mode":"MOCK" reveals testing mode

CORS preflight x-mastra-client-type reveals Mastra framework

Google OAuth Client ID 915542820771-nj20rj...

/api/audits POST error Required fields: audit_code, title, audit_type

Recommendation:

Remove all environment variable names from error messages. Return generic error messages (e.g., 'Service not configured') instead of specific variable names. Remove the 'mode' field from API responses. Suppress framework-identifying headers. Restrict Google OAuth Client ID visibility by using server-side token exchange only.

Reference:

OWASP Information Disclosure:

https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/08-Testing_for_Error_Handling/ |

CWE-209: Generation of Error Message Containing Sensitive Information: <https://cwe.mitre.org/data/definitions/209.html>

VULN-07: Insecure Session Cookie Configuration

HIGH

CVSS 7.4 | A02:2021 Cryptographic Failures

The walaplus_session cookie lacks critical security flags.

Flag Status Risk

HttpOnly MISSING XSS can steal session via document.cookie

Secure MISSING Cookie sent over HTTP (downgrade attack)

SameSite MISSING CSRF attacks possible

Path MISSING Cookie sent to all paths

Impact: Combined with Stored XSS (VULN-04), enables session hijacking via document.cookie.

Recommendation:

Set the following cookie flags on walaplus_session: HttpOnly (prevents JavaScript access), Secure (HTTPS-only transmission), SameSite=Lax (CSRF protection), Path=/ (or more restrictive). Example: Set-Cookie: walaplus_session=<token>; HttpOnly; Secure; SameSite=Lax; Path=/

Reference:

OWASP Session Management Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

| CWE-614: Sensitive Cookie in HTTPS Session Without 'Secure' Attribute: <https://cwe.mitre.org/data/definitions/614.html> | CWE-1004:

Sensitive Cookie Without 'HttpOnly' Flag: <https://cwe.mitre.org/data/definitions/1004.html>

VULN-08: JWT Role Claim in Token Payload

HIGH

CVSS 6.5 | A02:2021 Cryptographic Failures

The JWT token embeds the user's role directly in the payload. While signature verification is currently functional, this design pattern is risky. If the signing key is compromised or weak, privilege escalation to admin is trivial.

Token tampering results:

- Role changed to admin: 401 (signature verified)
- Empty signature: 401 (signature required)
- userId changed (IDOR): 401 (signature verified)

Recommendation:

Store the user role exclusively server-side (in the database) and look it up by userId on each request. Remove the role claim from the JWT payload. Use a standard JWT format with a proper header (alg, typ). Rotate signing keys periodically and use strong keys (256-bit minimum for HMAC-SHA256).

Reference:

OWASP JWT Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html |

CWE-565: Reliance on Cookies without Validation and Integrity Checking: <https://cwe.mitre.org/data/definitions/565.html> | RFC 7519

JSON Web Token: <https://datatracker.ietf.org/doc/html/rfc7519>

VULN-09: Missing Input Validation and Sanitization

HIGH

CVSS 7.3 | A03:2021 Injection

No input validation or sanitization is performed on any API input field.

Payload Type Field Stored Value

XSS risk_title <script>alert(document.cookie)</script>

SSTI probe risk_owner {{7*7}} (stored literally)

Proto pollution __proto__ Accepted in request body

Node.js RCE vendor.name {{constructor.constructor(...)()}}

Cmd injection vendor.email \${require('child_process').exec('id')}

Recommendation:

Implement strict input validation using a schema validation library (e.g., Joi, Zod, or Ajv for Node.js). Define explicit schemas for every API endpoint that specify allowed fields, types, and constraints. Sanitize HTML from all text inputs using DOMPurify server-side. Reject any input containing HTML tags, template expressions, or known injection patterns.

Reference:

OWASP Input Validation Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html | CWE-20:

Improper Input Validation: <https://cwe.mitre.org/data/definitions/20.html>

VULN-10: Mass Assignment / Unvalidated Parameters

HIGH

CVSS 6.5 | A04:2021 Insecure Design

API endpoints accept arbitrary extra parameters in POST/PUT bodies without validation, including __proto__ and constructor.prototype payloads.

Parameter Behavior

id: 999 Ignored (server assigns own ID)

created_by: admin Accepted, stored as null

role: admin Accepted in request

__proto__ Accepted without error

constructor.prototype Accepted without error

Recommendation:

Implement a strict allowlist of accepted fields for each API endpoint. Use an ORM with explicit field selection (e.g., Drizzle or Prisma select/pick). Strip any unrecognized fields from the request body before processing. Specifically filter out __proto__, constructor, and prototype keys to prevent prototype pollution.

Reference:

OWASP Mass Assignment: https://cheatsheetseries.owasp.org/cheatsheets/Mass_Assignment_Cheat_Sheet.html | CWE-915:

Improperly Controlled Modification of Dynamically-Determined Object Attributes: <https://cwe.mitre.org/data/definitions/915.html> |

Prototype Pollution: <https://portswigger.net/web-security/prototype-pollution>

VULN-11: No Rate Limiting on API Endpoints

MEDIUM

CVSS 5.3 | A04:2021 Insecure Design

No rate limiting was observed on any endpoint during testing. Dozens of requests were sent in rapid succession without throttling. Enables brute-force attacks, data exfiltration, and potential DoS.

Recommendation:

Implement rate limiting using middleware such as express-rate-limit for Node.js. Apply tiered limits: 100 requests/minute for read endpoints and 10-20 requests/minute for write endpoints per IP. Add exponential backoff on authentication failures. Consider using a WAF (Web Application Firewall) like Cloudflare or AWS WAF for DDoS protection.

Reference:

OWASP Rate Limiting: https://cheatsheetseries.owasp.org/cheatsheets/Denial_of_Service_Cheat_Sheet.html | CWE-770: Allocation of

Resources Without Limits: <https://cwe.mitre.org/data/definitions/770.html> | express-rate-limit:

<https://www.npmjs.com/package/express-rate-limit>

VULN-12: Admin API Key Authentication Pattern

MEDIUM

CVSS 5.9 | A07:2021 Identification & Auth Failures

The /api/users endpoint requires an Admin API key but the key delivery mechanism and strength are unknown. Error messages reveal the authentication mechanism.

```
GET /api/users -> {"error":"Unauthorized - Admin API key required"}
GET /api/users/1 -> {"error":"Unauthorized - Admin API key required"}
```

Recommendation:

Use a cryptographically random API key with minimum 256-bit entropy. Transmit via the Authorization header (Bearer scheme) over HTTPS only. Return a generic 403 Forbidden without revealing the authentication mechanism. Implement key rotation policy and store hashed API keys (bcrypt/argon2), not plaintext.

Reference:

OWASP API Key Management: https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html | CWE-798: Use of

Hard-coded Credentials: <https://cwe.mitre.org/data/definitions/798.html>

VULN-13: Unauthenticated Audit Record Creation

MEDIUM

CVSS 5.3 | A01:2021 Broken Access Control

The audit creation endpoint processes unauthenticated requests and reveals required fields through error messages.

```
POST /api/audits (missing fields) -> 400: {"error":"Missing required fields: audit_code, title, audit_type"}
POST /api/audits (with fields) -> 500: {"error":"Failed to create audit"}
```

Recommendation:

Apply authentication middleware to the /api/audits POST route before any business logic or input validation runs. Return a generic 401 Unauthorized before processing the request body. Suppress field-level error details for unauthenticated requests.

Reference:

OWASP Broken Access Control: https://owasp.org/Top10/A01_2021-Broken_Access_Control/ | CWE-306: Missing Authentication for Critical Function: <https://cwe.mitre.org/data/definitions/306.html>

VULN-14: OAuth State Parameter Not Validated

MEDIUM

CVSS 5.4 | A07:2021 Identification & Auth Failures

The OAuth callback processes requests with arbitrary state parameters without validating against the stored cookie state. The failure occurred at the token exchange step, not at state validation.

Recommendation:

Generate a cryptographically random state parameter (minimum 128-bit), store it in the oauth_state cookie (with Secure, HttpOnly, SameSite=Lax flags), and strictly validate it in the callback handler before proceeding with the token exchange. Reject the callback immediately if the state does not match.

Reference:

OWASP OAuth Security: https://cheatsheetseries.owasp.org/cheatsheets/OAuth2_Cheat_Sheet.html | RFC 6749 Section 10.12 - CSRF Protection: <https://datatracker.ietf.org/doc/html/rfc6749#section-10.12> | CWE-352: Cross-Site Request Forgery: <https://cwe.mitre.org/data/definitions/352.html>

VULN-15: Missing Security Headers

MEDIUM

CVSS 5.3 | A05:2021 Security Misconfiguration

Header Status Risk

Content-Security-Policy MISSING XSS not mitigated

X-Content-Type-Options MISSING MIME-sniffing possible

X-Frame-Options MISSING Clickjacking possible

X-XSS-Protection MISSING Browser XSS filter not activated

Strict-Transport-Security PRESENT OK (63072000s)

Referrer-Policy MISSING Referrer leakage

Recommendation:

Add the following security headers to all responses: Content-Security-Policy (script-src 'self'; object-src 'none'), X-Content-Type-Options: nosniff, X-Frame-Options: DENY, X-XSS-Protection: 1; mode=block, Referrer-Policy: strict-origin-when-cross-origin. Use the Helmet.js middleware for Node.js/Express to set these automatically.

Reference:

OWASP Security Headers: https://cheatsheetseries.owasp.org/cheatsheets/HTTP_Headers_Cheat_Sheet.html | Helmet.js: <https://helmetjs.github.io/> | CWE-693: Protection Mechanism Failure: <https://cwe.mitre.org/data/definitions/693.html> | securityheaders.com for validation

VULN-16: Verbose Error Messages

LOW

CVSS 3.7 | A05:2021 Security Misconfiguration

Error responses reveal internal implementation details including environment variable names, required field names, and endpoint existence.

Recommendation:

Implement a centralized error handler that maps internal errors to generic user-facing messages. Never expose environment variable names, stack traces, or internal field names in production responses. Use structured logging (e.g., Winston, Pino) to log detailed errors server-side only.

Reference:

OWASP Error Handling: https://cheatsheetseries.owasp.org/cheatsheets/Error_Handling_Cheat_Sheet.html | CWE-209: Generation of Error Message Containing Sensitive Information: <https://cwe.mitre.org/data/definitions/209.html>

VULN-17: Google OAuth Client ID Exposed

LOW

CVSS 3.1 | A05:2021 Security Misconfiguration

OAuth client ID exposed in authorization URLs:
915542820771-nj20rj53742aqdol4u696skcu1upsqrl.apps.googleusercontent.com. Domain restriction observed: walaplus.com.

Recommendation:

While OAuth Client IDs are semi-public by design, ensure strict redirect URI validation in the Google Cloud Console. Only allow exact-match redirect URIs (no wildcards). Verify the authorized JavaScript origins are limited to production domains only. Consider using server-side OAuth flow (Authorization Code with PKCE) to minimize client-side exposure.

Reference:

Google OAuth Security: <https://developers.google.com/identity/protocols/oauth2/web-server#protect-authorization-code> | CWE-200: Exposure of Sensitive Information: <https://cwe.mitre.org/data/definitions/200.html>

VULN-18: Mastra Framework Header Disclosure

LOW

CVSS 3.1 | A05:2021 Security Misconfiguration

CORS preflight response reveals x-mastra-client-type header, confirming the application uses the Mastra AI framework. This enables targeted exploit research.

Recommendation:

Remove the x-mastra-client-type header from all server responses. Configure the Mastra framework (or your server middleware) to suppress framework-identifying headers in production. Apply the same principle to all 'X-Powered-By' style headers (e.g., `app.disable('x-powered-by')` in Express).

Reference:

OWASP Security Misconfiguration: https://owasp.org/Top10/A05_2021-Security_Misconfiguration/ | CWE-200: Exposure of Sensitive Information: <https://cwe.mitre.org/data/definitions/200.html>

VULN-19: Potential Prototype Pollution via JSON Body

HIGH

CVSS 7.5 | A03:2021 Injection

POST endpoints accept `__proto__` and `constructor.prototype` properties in JSON bodies without sanitization. If server-side code uses `Object.assign()` or spread operators, this could enable prototype pollution leading to RCE or authentication bypass.

Evidence:

```
POST /api/risks
{"risk_title":"ProtoPolTest",__proto__":{"isAdmin":true},
 "constructor":{"prototype":{"isAdmin":true}}}
```

Response: 200 OK (record created successfully)

Recommendation:

Sanitize JSON request bodies by recursively stripping keys matching `__proto__`, `constructor`, and `prototype` before processing. Use `Object.create(null)` for safe objects or use `Map` instead of plain objects. Adopt a validation library (Ajv, Zod) with strict mode that rejects unknown properties. Consider using `--frozen-intrinsics` Node.js flag in production.

Reference:

Prototype Pollution - PortSwigger: <https://portswigger.net/web-security/prototype-pollution> | CWE-1321: Improperly Controlled Modification of Object Prototype Attributes: <https://cwe.mitre.org/data/definitions/1321.html> | Node.js Prototype Pollution Mitigation: <https://github.com/nicknisi/prototype-pollution-defense>

4. Test Coverage Matrix

Category	Techniques	Result
Authentication	JWT decode/tamper, session replay	JWT verified; API auth FAIL
Authorization	Unauth access, admin key bypass	CRITICAL: 12+ unprotected
XSS	Stored XSS in 6+ fields, 3 modules	CRITICAL: Confirmed
SQL Injection	UNION, boolean, time-based, stacked	PASS: Parameterized
SSTI	{{7*7}}, \${7*7}, constructor	PASS: Not executed
NoSQL Injection	\$ne, \$gt, \$regex operators	PASS: Not parsed
Proto Pollution	__proto__, constructor.prototype	CONCERN: Accepted
CORS	Wildcard origin, preflight, cross-PUT	CRITICAL: Wildcard
SSRF	Collaborator URLs in fields	PASS: No OOB
IDOR	Sequential IDs, JWT userId	Mixed: API open, JWT ok
Mass Assignment	Extra fields in POST body	CONCERN: Accepted
Rate Limiting	Rapid sequential requests	FAIL: No throttling

5. Burp Suite Evidence Index

#	Tab Name	Attack Type
1-23	Initial test scenarios	Phase 1 comprehensive tests
24	JWT-Tamper-Admin-Role	JWT role escalation attempt
25	JWT-Tamper-NoSignature	JWT signature bypass attempt
26	NEW-API-Dashboard	Unauthenticated dashboard access
27	NEW-API-Agents-Performance	Unauthenticated agent data
28	CRITICAL-Unauth-Risk-Tamper	Live data modification confirmed
29	CRITICAL-Unauth-Dashboard-Leak	Dashboard information disclosure
30	CRITICAL-Unauth-Agents-Leak	Agent metrics disclosure
31	INFO-Zoho-Cred-Leak	Credential variable name leak
32	JWT-Token-Analysis	Session token testing
33	SSTI-Prototype-Pollution	Injection and pollution testing
34	Unauth-Audit-Create	Audit record injection attempt
35	Unauth-Scorecards-Read	Unauthenticated scorecard access
36	OAuth-Callback-Manipulation	OAuth CSRF testing
37	CRM-Module-Enum	CRM module enumeration
38	CORS-Deep-Test	Cross-origin validation testing

6. Appendix: Raw Evidence Timestamps

Evidence Tab HTTP Status Timestamp

Unauth Dashboard Read #29 200 OK 2026-03-11T12:08:33Z

Unauth Agents Read #30 200 OK 2026-03-11T12:08:32Z

Risk Tamper (PUT id=1) #28 200 OK 2026-03-11T12:12:40Z

JWT Admin Tamper #24 401 Unauth 2026-03-11T12:08:26Z

JWT Empty Sig #25 401 Unauth 2026-03-11T12:08:30Z

JWT userId IDOR N/A 401 Unauth 2026-03-11T12:11:28Z

CORS Preflight #38 204 2026-03-11T12:13:37Z

Zoho Config Leak #31 400 Bad Req 2026-03-11T12:08:49Z

Audit Field Enum #34 400 Bad Req 2026-03-11T12:12:16Z

Valid Session Test #32 200 OK 2026-03-11T12:13:40Z

Report generated: March 11, 2026

Testing tools: Burp Suite Professional, Browser Testing, Manual Analysis

Methodology: OWASP Testing Guide v4.2, OWASP Top 10 2021